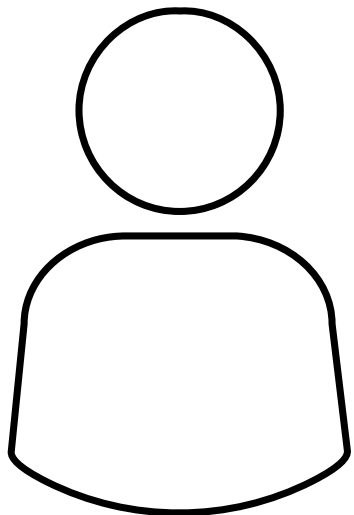
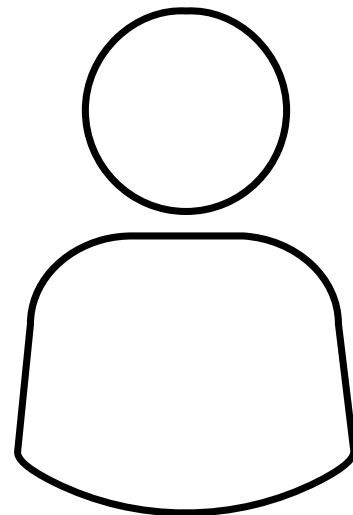


## INTEGRANTES

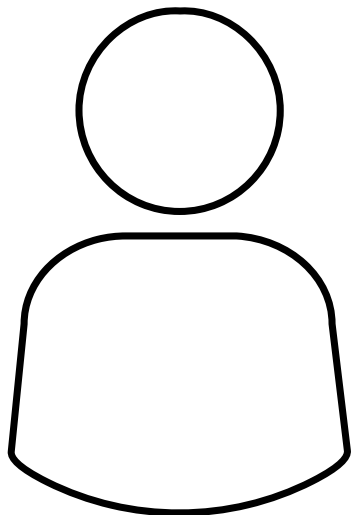


Flores Olivares  
Omar

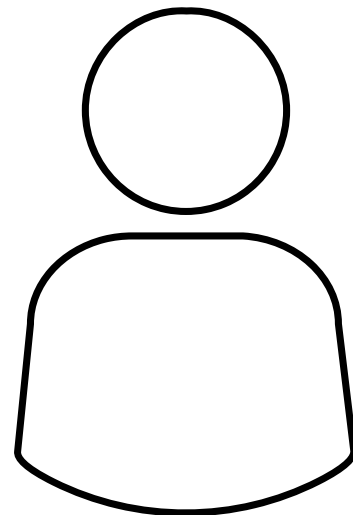


Roa Venegas  
Brenda

## INTEGRANTES



**Flores Olivares**  
**Omar**



**Roa Venegas**  
**Brenda**

- Proyecto Propio (ERGON)
  - Caso de estudio
    - Entrevista Simulada
  - Modelo ER
  - Modelo EER
  - Modelo Relacional
  - Landing Page
- Proyecto Asignado
  - Proposito
  - Version estatica GitHub Pages
  - Ejecucion de version con backend
- Articulos
  - Arquitectura de SGBD + Inteligencia Artificial / Aprendizaje Automatico
  - Arquitectura y Clasificacion de SGBD Analiticos Modernos
  - Arquitectura Interna del SGBD y Sistemas Operativos

Proyecto Propio

# ERGON

# Caso de estudio

La empresa **ERGON**, perteneciente al sector industrial de especialidades petroleras y aceites, expresó a través de una entrevista con su *Gerente Comercial* la necesidad de un sistema centralizado para la gestión integral de sus pedidos y comercialización de productos.

Por lo que, con el fin de cumplir con todos sus requerimientos, se propuso la consolidación en una base de datos de la información relativa a su catálogo de aceites industriales y cartera de clientes B2B, garantizando el registro detallado de transacciones, el cálculo automático de indicadores financieros y la generación de reportes comerciales precisos.

# Entrevista simulada

## *Consultor de BD y Gerente Comercial de Ergon*

- **Consultor:** ¿Qué datos clave necesitan registrar sobre los productos de aceites y cómo se identifican?
- **Gerente:** Manejamos líneas especializadas como Hyvolt (y sus variantes), Omnigold y Hyprene. Cada producto tiene una clave numérica única de 7 dígitos. Necesitamos guardar su nombre, precio unitario de venta y costo de producción para calcular el margen de utilidad.
- **Consultor:** ¿Cómo manejan la información de los clientes?
- **Gerente:** Nuestros clientes son otras empresas que transforman nuestros insumos. A cada uno le asignamos un número único de cliente de 6 dígitos. Guardamos su razón social, datos de contacto e identificación fiscal.
- **Consultor:** ¿Qué información se genera en las transacciones de venta?
- **Gerente:** Cada pedido registra qué cliente compra, los productos adquiridos, las cantidades, la fecha de transacción y el monto total en dólares.
- **Consultor:** ¿Con qué frecuencia consultan la base de datos y qué reportes requieren?
- **Gerente:** La consulta de ventas es diaria para seguimiento operativo. Los reportes consolidados son mensuales y anuales: necesitamos conocer los ingresos totales en dólares, la utilidad neta acumulada, el ranking de productos más vendidos y los clientes que generan mayor volumen de compra.

# Modelo Entidad Relacion (ER)

## ENTIDADES

**CLIENTE**

**PEDIDO**

**DETALLE**

**PRODUCTO**

### **CLIENTE - PEDIDO (realiza) · 1:N**

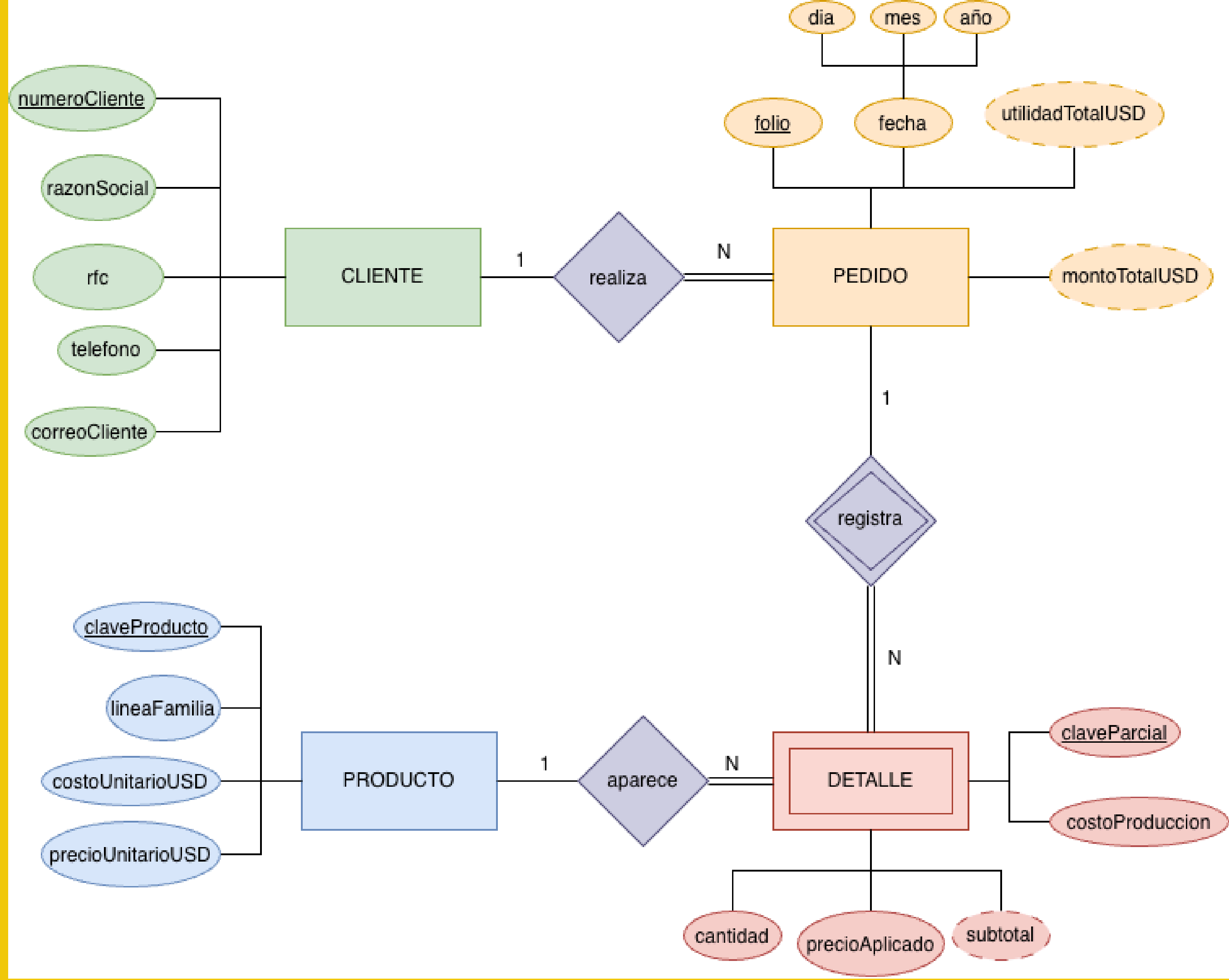
Un cliente puede hacer muchos pedidos. Cada pedido lo realiza un único cliente.

### **PEDIDO - DETALLE (registra) · 1:N**

Un pedido puede tener muchos detalles. Cada detalle pertenece a un solo pedido y depende de él para existir y para identificarse, por eso DETALLE es una entidad débil.

### **PRODUCTO - DETALLE (aparece) · 1:N**

Un producto puede aparecer en muchos detalles. Cada detalle se refiere a un único producto.



# Modelo Entidad Relacion Extendido (EER)

Partiendo del modelo entidad-relación, como requisitos extendidos se agregan 2 entidades: **SUCURSAL** y **TRANSPORTISTA**. Esto con el fin de poder representar la logistica de ERGON.

## **CLIENTE - PEDIDO (realiza) · 1:N**

Un cliente puede no haber hecho ningún pedido o haber hecho muchos (0,N). Cada pedido lo realiza un único cliente y no existe sin él (1,1).

## **CLIENTE - SUCURSAL (tiene) · 1:N**

Un cliente tiene al menos una planta o almacén de recepción y puede tener varias (1,N). Cada sucursal pertenece a un solo cliente, que es quien la identifica (1,1).

## **PEDIDO - DETALLE (registra) · 1:N**

Un pedido tiene al menos un detalle y puede tener muchos (1,N). Cada detalle pertenece a un solo pedido y depende de él para existir y para identificarse (1,1).

## **PRODUCTO - DETALLE (aparece) · 1:N**

Un producto del catálogo puede no haberse vendido aún o aparecer en muchos detalles (0,N). Cada detalle se refiere a un único producto (1,1).

### **PEDIDO - SUCURSAL - TRANSPORTISTA (envio) · Ternaria**

Un pedido tiene como máximo un envío, y puede no tenerlo todavía (0,1). Una sucursal puede no haber recibido envíos o recibir muchos (0,N). Un transportista puede no haber realizado envíos o realizar muchos (0,N).

### **Especializaciones**

#### **CLIENTE:**

Disjunta y total. Todo cliente es nacional o de exportación, pero no ambos.

#### **PRODUCTO:**

Disjunta y total. Cada producto pertenece a una sola línea comercial: Hyvolt, Omnigold o Hyprene.

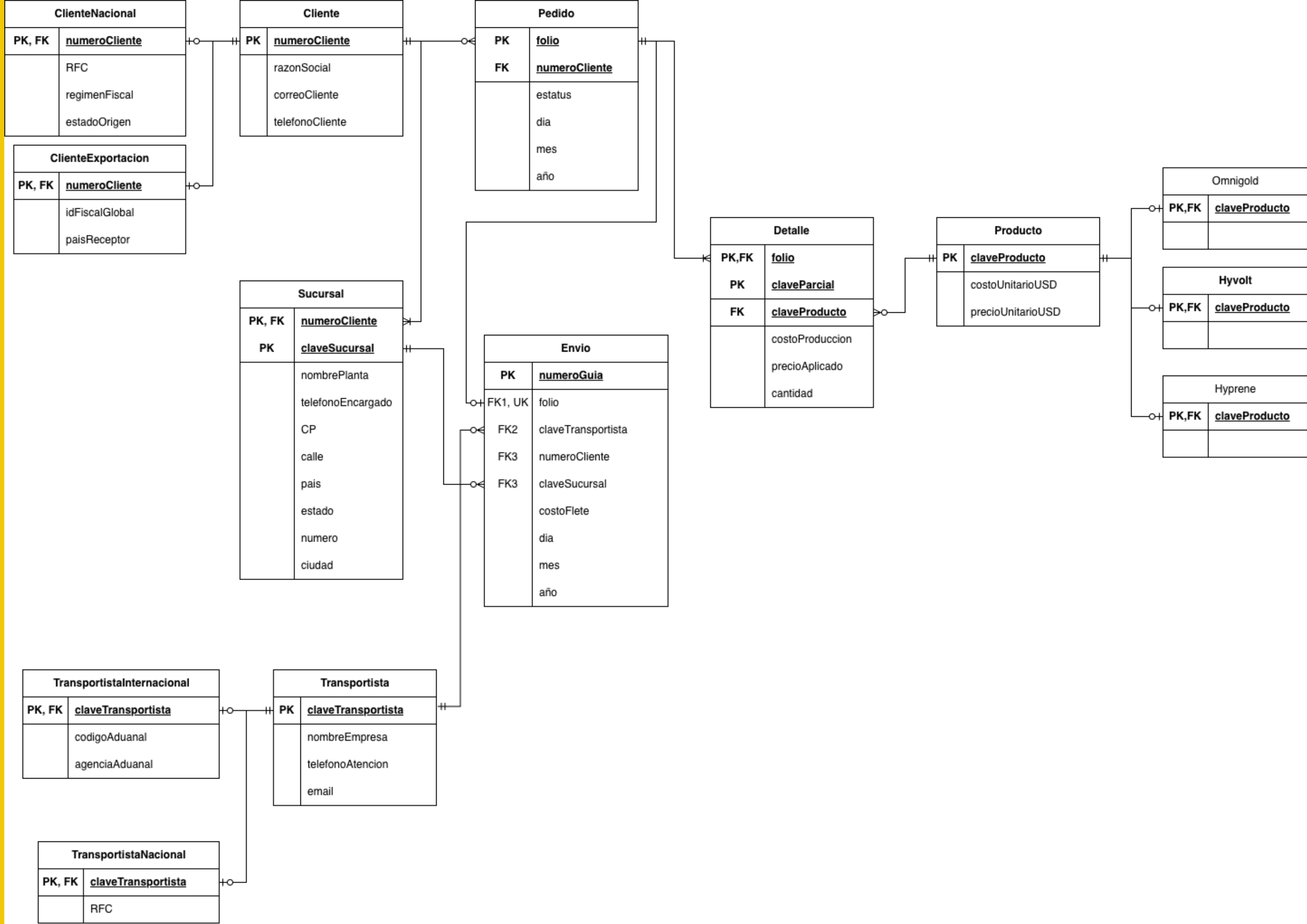
#### **TRANSPORTISTA:**

Solapada y total. Toda empresa es nacional o internacional, y una internacional también puede operar dentro del país.



# Modelo Relacional

Con el modelo entidad - relacion extendido, podemos trazar su modelo relacional. El cual denota las claves primarias y foraneas que le proporcionan al diagrama la capacidad de ser adaptable a codigo SQL.



# Proyecto Asignado



# Arquitectura de SGBD + Inteligencia Artificial / Aprendizaje Automatico

## RESUMEN

El artículo propone mejorar la indexación de bases de datos usando Machine Learning. Los índices tradicionales, como B-trees y tablas hash, pueden presentar problemas cuando los datos crecen mucho y se necesitan búsquedas más rápidas.

Los autores plantean utilizar modelos predictivos como índices, capaces de aprender la distribución de los datos y predecir dónde se encuentra cada información. Estos modelos pueden ser más rápidos y ocupar menos espacio cuando los datos tienen patrones claros, pero funcionan peor con datos irregulares o caóticos.

Por ello, proponen un enfoque híbrido, combinando modelos de ML con estructuras tradicionales para adaptarse a diferentes tipos de datos.

Aporte al proyecto: muestra cómo el Machine Learning puede optimizar el almacenamiento y las consultas en bases de datos, superando en ciertos casos las limitaciones de los índices convencionales.

Cita APA (7ª ed.) con DOI: Kraska, T., Beutel, A., Chi, E. H., Naughton, J., & Polyzotis, N. (2018). The case for learned index structures. En Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18) (pp. 489–504). Association for Computing Machinery. <https://doi.org/10.1145/3183713.3196909>

# Arquitectura y Clasificación de SGBD Analíticos Modernos

## RESUMEN

El artículo presenta DuckDB, un motor de base de datos embebido diseñado para realizar análisis de datos de forma rápida y sencilla, sin necesitar una infraestructura pesada como PostgreSQL o Spark.

DuckDB utiliza una arquitectura orientada a columnas y técnicas como vectorized execution, lo que permite procesar grandes cantidades de datos de manera eficiente.

Los resultados muestran que puede ejecutar consultas analíticas mucho más rápido que SQLite y utilizando menos recursos que otros motores tradicionales.

Aporte al proyecto: ayuda a entender cuándo conviene utilizar una base de datos embebida como DuckDB frente a un sistema más pesado como PostgreSQL, considerando el rendimiento y el consumo de recursos.

Cita APA (7ª ed.) con DOI: Raasveldt, M., & Mühleisen, H. (2019). DuckDB: An embeddable analytical database. En Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19) (pp. 1981–1984). Association for Computing Machinery. <https://doi.org/10.1145/3299869.3320212>

# Arquitectura Interna del SGBD y Sistemas Operativos

## RESUMEN

El artículo propone Paetica, una arquitectura para que OceanBase funcione eficientemente tanto en un solo servidor como en sistemas distribuidos. Su objetivo es evitar sobrecargas innecesarias en instalaciones pequeñas y, al mismo tiempo, permitir escalar cuando aumentan las necesidades.

Paetica combina características de arquitecturas Shared-Nothing y Shared-Everything, adaptándose según el entorno: en modo individual reduce la sobrecarga, mientras que en modo cluster permite procesamiento paralelo y escalabilidad.

Los resultados muestran que OceanBase con Paetica mejora el rendimiento y escala de forma eficiente, superando a sistemas como MySQL y Greenplum en algunas pruebas.

Aporte al proyecto: permite comprender la importancia de diseñar SGBD flexibles, escalables y eficientes, además de destacar el papel de la integridad ACID y los modelos relacionales en sistemas modernos.

Cita APA (7ª ed.) con DOI: Suresh, L., Yu, X., Zaharia, M., & Stonebraker, M. (2023). DBOS: A database-oriented operating system. *Proceedings of the VLDB Endowment*, 16(11), 3085–3097. <https://doi.org/10.14778/3611540.3611560>